

Simulazione di un sistema a N-corpi

Casadio Samuele e Facchini Diego

Ultima modifica: 27/08/2026

1 Scelte progettuali e implementative

Abbiamo diviso il codice in due macroparti ovvero quella grafica e quella di calcolo. Queste due parti sono a loro volta divise in un file di intestazione (header), dove vengono dichiarate le classi e le funzioni, e in un file di implementazione, dove vengono definite le funzioni dichiarate nell'header. Entrambe le parti vengono poi implementate in un file denominato `run.cpp`, dal quale è possibile visionare la simulazione

Per la parte di calcolo abbiamo deciso di fare varie funzioni per il calcolo delle componenti usate nella generazione dei grafici e le abbiamo raggruppate in una singola funzione principale che le contenesse tra loro.

La scelta è stata fatta per mantenere maggiore ordine e chiarezza all'interno del codice. Ad esempio, il calcolo delle accelerazioni è troppo complicato per poter stare all'interno della funzione dell'algoritmo velocity verlet, di conseguenza averlo in una funzione separata garantisce maggiore chiarezza sia nella lettura che nella struttura logica del codice.

2 Costrutti non introdotti a lezione

I costrutti non introdotti a lezione, usati nel nostro progetto, corrispondono a tutti quelli necessari per il corretto funzionamento della parte grafica con l'uso della libreria esterna SFML.

3 Librerie esterne

L'unica libreria esterna usata nel progetto è quella di SFML (Simple and Fast Multimedia Library), utilizzata per poter generare tutta la parte grafica del progetto, ovvero la generazione dei grafici e la rappresentazione visiva della simulazione.

Per installare la libreria su Linux è possibile utilizzare il comando: `sudo apt install libsFML-dev`

4 Compilazione ed esecuzione

Per compilare ed avviare il programma è necessario usare CMake. È sufficiente eseguire le seguenti righe di codice dal terminale, eseguendo prima il comando presente nella sezione 3 nel caso non si siano già installate le librerie usate.

```
cmake -S . -B build
cmake --build build --config Release
./build/Release/run config.txt
```

La prima riga configura il progetto, generando i file necessari alla compilazione nella cartella `build`.

La seconda permette la compilazione di tutti i file `run.cpp` `main.cpp` `graphics.cpp`, generando

l'eseguibile `run` all'interno della cartella `Release`.

L'esecuzione della terza riga permette di avviare la simulazione a partire dai dati presenti nel file `.txt` dato in input (si veda 5.1 per il formato da utilizzare).

Se si vogliono invece compilare ed eseguire i test sulla parte di calcolo da noi eseguiti, basta eseguire le seguenti righe di codice dal terminale:

```
cmake --build build --config Debug
ctest --test-dir build -C Debug
```

In questo caso, la prima riga compila anche l'eseguibile dedicato ai test, mentre la seconda li esegue tramite `CTest`, riportandone l'esito nel terminale.

5 Input e output

5.1 Input

I dati forniti in input al programma sono i seguenti:

- Numero di corpi
- Numero totale di step da eseguire
- Masse dei singoli corpi
- Raggi dei singoli corpi
- Posizioni iniziali dei singoli corpi
- Velocità iniziali dei singoli corpi

Il file `.txt` dato in input deve essere rigorosamente strutturato nel seguente formato, dividendo ogni singolo valore nella stessa riga con uno spazio:

- Prima riga: [numero di corpi] [numero di step]
- Seconda riga: [massa] di ogni corpo
- Terza riga: [raggio] di ogni corpo
- Quarta riga: [posizione x] [posizione y] per ogni corpo
- Quinta riga: [velocità x] [velocità y] per ogni corpo

5.2 Output

L'output è composto da due finestre diverse, una contenente la simulazione grafica e l'altra avente tre grafici in funzione del numero di step: uno del momento angolare totale, uno della quantità di moto totale e uno dell'energia totale.

6 Esempi con interpretazione dei risultati

Tutti i seguenti file .txt sono già presenti all'interno della cartella Esempi.

L'esempio della `Figure8` presenta un'orbita speciale scoperta da Chenciner e Montgomery nel 2000. Il sistema è composto da tre corpi con masse uguali con coordinate iniziali predefinite.

```
./build/Release/run Esempi/figure8.txt
```

Quello che è osservabile dalla simulazione è la forma di un 8, data dalle orbite dei corpi. Si può notare che data l'assenza di collisioni, è verificabile la conservazione di quantità di moto e momento angolare.

L'esempio `HeadOnCollision` presenta la collisione di due corpi aventi masse uguali e velocità opposte e con lo stesso modulo.

```
./build/Release/run Esempi/HeadOnCollision.txt
```

Dalla simulazione è possibile osservare la collisione di due corpi. A seguito della collisione si può osservare un unico corpo con velocità nulla. Quantità di moto totale e momento angolare totale si conservano rimanendo uguali a 0. L'energia totale, invece, subisce una variazione, riducendosi a 0 dopo l'urto. A seguire, abbiamo realizzato altri file di configurazione che ci risultavano rappresentativi di certe orbite o sistemi notevoli. In primo luogo, non poteva mancare una tipologia di Sistema solare.

```
./build/Release/run Esempi/SolarSystem.txt
```

La configurazione presenta 5 corpi di raggio ridotto (Pianeti) ruotanti attorno ad un corpo centrale più grande (Stella). Essendo un sistema stabile nel tempo (Orbite circolari), i momenti lineari e angolari totali rimangono costanti (come risulta nella finestra contenente i grafici).

Condizioni simili di momenti possono essere osservati in altri esempi, eseguibili tramite:

```
./build/Release/run Esempi/BinarySystem.txt
```

e

```
./build/Release/run Esempi/SymmetricOrbit4.txt
```

Si può accedere anche ad altri esempi inserendo i comandi:

```
./build/Release/run Esempi/GentleFusion.txt
```

e

```
./build/Release/run Esempi/SlowEscape.txt
```

7 Strategia di test

La strategia di test principale è stata quella di dividere i test della parte di calcolo da quelli della parte grafica e di input. In 4 è illustrato come eseguire i test sulla parte di calcolo. Tali test sono strutturati per testare il comportamento degli algoritmi nei casi standard e nei casi limite. I casi riguardano il numero di corpi, le collisioni e le conservazioni di energia, quantità di moto e momento angolare.

Per testare la parte grafica e di input invece abbiamo fatto una serie di test illustrati in 6. Nella medesima sezione vengono riportati gli aspetti verificati e i comportamenti attesi per ciascun test.

8 Uso di Intelligenza Artificiale generativa

L'utilizzo dell'Intelligenza Artificiale generativa è stato molto limitato. Per quanto riguarda la creazione del codice, l'unico caso in cui l'abbiamo sfruttata è stato nel file `run.cpp`, nel quale è stata utilizzata per rendere il programma di poter leggere un file in input con qualsiasi nome, in formato `.txt`.

Infine abbiamo utilizzato l'IA per ottenere chiarimenti riguardo il funzionamento delle librerie SFML, senza la generazione di alcun tipo di codice.